

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



**Lê Quang Hồng Ngọc - 52300135
Nguyễn Trí Nhân - 52300138**

BÁO CÁO CUỐI KỲ NHẬP MÔN XỬ LÝ ẢNH SỐ

**Người hướng dẫn
TS. Nguyễn Chí Thiện**

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025

LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn thầy Nguyễn Chí Thiện, giảng viên bộ môn Xử lý ảnh số, người đã luôn tận tình hướng dẫn, truyền đạt và giảng dạy cũng như cung cấp cho chúng em nhiều kiến thức chuyên ngành thú vị giúp chúng em định hình và hoàn thành đồ án một cách tốt nhất.

Đồng thời, chúng em cũng xin bày tỏ lòng biết ơn sâu sắc đến Ban Giám hiệu và đoàn thể trường nói chung đã tạo điều kiện tối ưu về cơ sở vật chất, giáo trình và môi trường học tập.

Một lần nữa chúng em xin chân thành cảm ơn

TP. Hồ Chí Minh, ngày 25 tháng 11 năm 2025

Tác giả

(Ký tên và ghi rõ họ tên)

Nhân

Nguyễn Trí Nhân

Ngọc

Lê Quang Hồng Ngọc

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của **TS. Nguyễn Chí Thiện**. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 25 tháng 11 năm 2025

Tác giả

(Ký tên và ghi rõ họ tên)

Nhân

Nguyễn Trí Nhân

Ngọc

Lê Quang Hồng Ngọc

TÓM TẮT

Báo cáo này tập trung trình bày và phân tích phương pháp giải quyết bài toán Tự động Phát hiện và Nhận dạng Biển báo Giao thông (Traffic Sign Detection and Recognition) trong môi trường video thực tế, thực nghiệm trên hai video đầu vào (video1.mp4 và video2.mp4). Mục tiêu là xây dựng một chuỗi xử lý ảnh để khoanh vùng chính xác biển báo và suy luận nội dung tương ứng.

Về mặt phương pháp, đề tài tập trung xây dựng một hệ thống nhận dạng biển báo dựa trên các nguyên tắc cơ bản của Xử lý Ảnh Số, tuân thủ nghiêm ngặt quy tắc không sử dụng mô hình huấn luyện sẵn. Chuỗi xử lý được thiết kế là sự kết hợp logic giữa các kỹ thuật xử lý ảnh căn bản (như Lọc màu, Tách ngưỡng) với các phương pháp nâng cao hơn (như Phân tích Contours, Khớp đặc trưng/mẫu) nhằm đạt hiệu suất cao và ổn định trong các điều kiện môi trường khác nhau.

MỤC LỤC

Contents

DANH MỤC HÌNH VẼ.....	vii
DANH MỤC BẢNG BIỂU.....	ix
DANH MỤC CÁC CHỮ VIẾT TẮT.....	x
CHƯƠNG 1. Cơ Sở Lý Thuyết.....	1
1.1 Không gian HSV	1
1.1.1 Lọc màu trong không gian HSV.....	1
1.1.2 Xác định và tinh chỉnh ngưỡng màu.....	2
1.2 Kỹ Thuật Tách Ngưỡng (Thresholding).....	3
1.2.1 Kỹ thuật tách ngưỡng cơ bản.....	3
1.2.2 Tách ngưỡng trong không gian HSV.....	4
1.3 Xử lý hình thái học (Morphology)	5
1.3.1 Phép co (erode) và Phép giãn (dilate)	5
1.3.2 Phép mở (opening) và phép đóng (closing).....	6
1.4 Tìm viền (Finding Countour).....	7
1.5 Phân tích độ tròn.....	9
1.6 Trích xuất vùng quan tâm (ROI)	10
1.7 Nhận diện biển báo bằng ORB Feature Matching	11
1.7.1 Trích xuất và mô tả đặc trưng bằng ORB.....	11
1.7.2 Nhận dạng biển báo bằng Feature Matching sử dụng ORB	12
1.8 Kết luận.....	13
CHƯƠNG 2. THỰC NGHIỆM VÀ KẾT QUẢ	14

2.1 Import và hàm circularity.....	14
2.2 Load các template và trích xuất đặc trưng ORB	15
2.2.1 Tạo cấu trúc lưu dữ liệu + ORB detector.....	16
2.2.2 Kiểm tra thư mục chứa templates.....	16
2.2.3 Duyệt từng thư mục biển báo.....	16
2.2.4 Lấy danh sách file ảnh trong từng loại.....	16
2.2.5 Trích xuất đặc trưng template.....	16
2.2.6 Lưu và trả về sau khi đã chạy qua hết.....	17
2.2.7 Kết luận khối code	17
2.3 So khớp đặc trưng.....	17
2.3.1 Kiểm tra đầu vào	18
2.3.2 Chuẩn hóa ROI.....	19
2.3.3 Trích xuất đặc trưng ROI.....	19
2.3.4 Matching với template	19
2.3.5 Kết luận.....	21
2.4 Hàm xử lý chính	21
2.4.1 Khởi tạo.....	21
2.4.2 Xử lý hình thái học	24
2.4.3 Loại nhiễu	25
2.5 Xử lý feature matching	26
2.6 Vẽ output.....	27
2.7 Kết quả thực nghiệm.....	28
CHƯƠNG 3. Kết luận và hướng phát triển	38

3.1 Những điều chưa đạt được trong đề tài	39
3.2 Những điều đã đạt được trong đề tài	39
3.3 Kết luận.....	39
TÀI LIỆU THAM KHẢO	40

DANH MỤC HÌNH VẼ

Figure 1. Import & Circularity	14
Figure 2. Load và trích xuất đặc trưng của template.....	15
Figure 3. Orb Classify	18
Figure 4. Init trong hàm main	22
Figure 5. Xử lý màu	24
Figure 6. Xử lý hình thái học.....	24
Figure 7. Loại nhiễu	25
Figure 8. Xử lý theo màu.....	26
Figure 9. Xử lý feature matching.....	27
Figure 10. Video 1 frame 1.....	29
Figure 11. Video 1 frame 2.....	29
Figure 12. Video 1 frame 3.....	30
Figure 13. Video 1 frame 4.....	30
Figure 14. Video 1 frame 5.....	31
Figure 15. Video 1 frame 6.....	31
Figure 16. Video 1 frame 7.....	32
Figure 17. Video 1 frame 8.....	32
Figure 18. Video 1 frame 9.....	33
Figure 19. Video 1 frame 10.....	33
Figure 20. Video 2 frame 1.....	34
Figure 21. Video 2 frame 2.....	34
Figure 22. Video 2 frame 3.....	35

Figure 23. Video 2 frame 4.....	35
Figure 24. Video 2 frame 5.....	36
Figure 25. Video 2 frame 6.....	36
Figure 26. Video 2 frame 7.....	37
Figure 27. Video 2 frame 8.....	37
Figure 28. Video 2 frame 9.....	38
Figure 29. Video 2 frame 10.....	38

DANH MỤC BẢNG BIỂU

Table 1. Phép mở và Phép đóng.....	6
Table 2. Thuộc tính của findContours.....	8
Table 3. Các biến khởi tạo	22

DANH MỤC CÁC CHỮ VIẾT TẮT

HSV	Hue - Saturation - Value
RGB	Red – Green - Blue
ROI	Region of Interest
ORB	Oriented FAST and Rotated BRIEF
kp	Keypoints
des	Descriptor

CHƯƠNG 1. Cơ Sở Lý Thuyết

Chương này sẽ tập trung vào phân tích các lý thuyết được sử dụng trong đề tài.

1.1 Không gian HSV

1.1.1 Lọc màu trong không gian HSV

a) Đặt vấn đề

Khi muốn tách các đối tượng với đặc trưng là màu sắc (đối với bài toán được nêu trong đề tài thì là các biển báo giao thông) ra khỏi nền nhiễu phức tạp. Cách tiếp cận tự nhiên nhất là dựa vào màu sắc của chúng.

Tuy nhiên, trong không gian màu sắc RGB, thông tin về màu sắc và độ sáng nhiều lúc sẽ bị trộn lẫn.

Ví dụ: Với một pixel màu đỏ $R = (200, 0, 0)$ khi bị bóng râm che khuất thì sẽ bị giảm giá trị đồng thời cả 3 kênh thành $R' = (100, 0, 0)$. Máy tính sẽ nhầm lẫn và coi đây là hai giá trị số học hoàn toàn khác nhau, dẫn đến việc bỏ sót.

⇒ RGB có tính tương quan cao giữa các kênh màu (Đỏ - Xanh - Lam), không đảm bảo tính ổn định khi điều kiện ánh sáng môi trường thay đổi liên tục.

b) Không gian màu HSV

Không gian màu HSV được sử dụng để khắc phục nhược điểm trên, để hiểu rõ hơn vì sao, ta sẽ phân tích các thành phần của không gian màu HSV:

- + Hue (Sắc độ): Biểu diễn loại màu (ví dụ: Đỏ, xanh, vàng). Đây là kênh quan trọng nhất vì nó bất biến với sự thay đổi của ánh sáng trắng
- + Saturation (Độ bão hòa): Biểu diễn độ tinh khiết của màu sắc (tránh các màu quá nhạt hoặc bị pha trắng)
- + Value (Giá trị): Biểu diễn cường độ sáng tối.

Bằng cách tách riêng kênh H, ta có thể định nghĩa màu “Đỏ” là một khoảng giá trị Hue cụ thể, bất kể pixel đó có bị thay đổi về sáng hay tối (Giá trị thay đổi).

Mở rộng: Theo lý thuyết, Hue có giá trị từ 0 độ tới 360 độ. Tuy nhiên, trong thư viện OpenCV (vì sử dụng kiểu dữ liệu uint8), giá trị Hue được nén xuống phạm vi [0,179] để vừa vặn trong 1 byte (256 giá trị). Việc xác định ngưỡng màu nên tuân thủ thang đo này.

c) Định nghĩa khoảng màu và vấn đề của màu đỏ

Đối với màu đỏ, ta gặp phải hiện tượng vòng lặp trên vòng màu:
+ Màu đỏ nằm ở điểm bắt đầu (0 độ) và điểm kết thúc (360 độ, hay 180 trong OpenCV)

+ Để giải quyết vấn đề trên, thay vì chỉ dùng một khoảng để định nghĩa màu đỏ, ta cần định nghĩa 2 khoảng riêng biệt cho màu đỏ trong OpenCV:

- Khoảng 1: Hue từ 170 đến 180
- Khoảng 2: Hue từ 0 tới 10

1.1.2 Xác định và tính chỉnh ngưỡng màu

a) Nguyên tắc

Trong thực tế, rất hiếm khi một vật thể có đúng một giá trị màu cố định. Các yếu tố như thời tiết, hướng chiếu sáng, bóng râm, chất lượng camera, nhiễu cảm biến... đều làm thay đổi giá trị đo được.

Vì vậy, ta **không thể chọn một giá trị duy nhất**, mà phải chọn một **khoảng giá trị chấp nhận được** cho từng kênh (H, S, V).

b) Tính chỉnh ngưỡng màu

Chỉ dùng mỗi kênh Hue là chưa đủ, vì:

- Nền cảnh hoặc các vật thể khác có thể có Hue tương tự (na ná cùng màu).
- Một số vùng có màu gần đúng nhưng rất nhạt hoặc bị pha trắng, không nên xem là đối tượng.

Do đó, khi xác định ngưỡng, ta cần:

- Đặt khoảng H cho loại màu quan tâm (ví dụ: hai khoảng cho màu đỏ như trên).
- Giới hạn khoảng S để loại bỏ các vùng màu quá nhạt (S quá thấp).

- Giới hạn khoảng V để loại bỏ các vùng quá tối hoặc quá sáng, không còn giữ được đặc trưng màu rõ ràng.

Quá trình thực tế thường gồm:

1. Chọn khoảng H, S, V ban đầu dựa trên kinh nghiệm hoặc tài liệu.
2. Quan sát kết quả mask trên nhiều khung hình / ảnh khác nhau.
3. Điều chỉnh dần các ngưỡng (đặc biệt là S và V) tới khi vùng biển báo được giữ lại tốt, còn nền và nhiễu được loại bỏ tối đa.

1.2 Kỹ Thuật Tách Ngưỡng (Thresholding)

1.2.1 Kỹ thuật tách ngưỡng cơ bản

a) Định nghĩa

Tách ngưỡng là một quá trình biến đổi một bức ảnh đầu vào (ảnh màu, ảnh xám hoặc mặt nạ) thành một ảnh nhị phân, nơi mỗi pixel chỉ có 2 giá trị (0 hoặc 255).

Nó hoạt động như một bộ lọc ra quyết định dứt khoát. Bằng cách so sánh giá trị cường độ của mỗi pixel với một giá trị ngưỡng T cố định.

Ta có thể hiểu thresholding theo nguyên tắc sau:

Ví dụ:

+ Ta có một ngưỡng T được xác định trước, pixel đầu vào mang một giá trị riêng của nó.

+ Nếu giá trị của pixel đầu vào $> T$, giá trị của pixel này sẽ là 255 (trắng)

+ Nếu giá trị của pixel đầu vào $< T$, giá trị của pixel này sẽ là 0 (đen)

b) Tách ngưỡng cơ bản dùng để làm gì?

Đây là bước đầu tiên để phân đoạn, nó giúp tách biệt một cách rõ ràng vùng tiền cảnh với hậu cảnh.

Việc làm này còn giúp làm giảm độ phức tạp của dữ liệu, tạo ra các ranh giới sắc nét cho các thuật toán hình học tiếp theo.

Tuy nhiên, thresholding cũng có giới hạn, tách ngưỡng cơ bản chỉ hiệu quả khi điều kiện ánh sáng trong toàn bộ bức ảnh là đồng nhất.

c) Cách hoạt động

Quá trình tách ngưỡng được thực hiện thông qua hàm `cv2.threshold` trong OpenCV với công thức chung đã đề cập ở phần trên:

$$g(x, y) = \begin{cases} 255 & \text{nếu } f(x, y) > T \\ 0 & \text{nếu } f(x, y) \leq T \end{cases}$$

Với:

+ $g(x, y)$ là giá trị một pixel của ảnh xám

+ $f(x, y)$ là giá trị cường độ

+ T là ngưỡng cố định

1.2.2 Tách ngưỡng trong không gian HSV

a) Định nghĩa

Đây là một dạng tách ngưỡng đa chiều, nơi ngưỡng T không còn là một con số cố định mà là một khối không gian màu được định nghĩa bởi Cận dưới (Lower) và Cận trên (Upper) trong không gian HSV.

Kết quả của quá trình tách ngưỡng này ta sẽ được một ảnh nhị phân gọi là Mặt nạ (mask), chỉ chứa các pixel thỏa mãn điều kiện màu sắc.

b) Tách ngưỡng trong không gian HSV

Tách ngưỡng trong không gian HSV có thể khắc phục được các nhược điểm mà tách ngưỡng cơ bản không thể.

Vì ta lọc dựa trên kênh H, kết quả tách ngưỡng hầu như không bị ảnh hưởng bởi sự thay đổi của độ sáng (Value/V) -> Giúp thuật toán ổn định trong điều kiện video phức tạp.

Đối với các bài toán có có màu sắc là đặc trưng mạnh nhất (trong trường hợp này là các biển báo). Kỹ thuật này cho phép ta chọn lọc và cô lập các vùng màu đỏ hoặc xanh dương với độ chính xác cao.

c) Cách hoạt động

Đối với tách ngưỡng trong không gian HSV, ta sẽ dùng hàm `cv2.inRange` của OpenCV với cơ chế so sánh trên từng kênh.

Ta có thể phân tích quá trình chuyển hóa như sau:

1. Duyệt pixel: Duyệt qua từng pixel

2. Kiểm tra: Pixel P chỉ được chọn nếu nằm trong vùng thỏa mãn của điều kiện HSV cho trước
3. Hợp nhất: Nếu sử dụng nhiều dải màu (như với màu Đỏ), nhiều Mask nhị phân sẽ được tạo ra và hợp nhất bằng phép toán Bitwise OR.

1.3 Xử lý hình thái học (Morphology)

Các phép toán Morphology được định nghĩa gốc trên ảnh nhị phân. Vì vậy phần giải thích dưới đây sử dụng mô hình foreground = 255 và background = 0.

a) Định nghĩa

Xử lý hình thái học là tập hợp các phép toán được áp dụng cho ảnh nhị phân được dùng để làm sạch, cải thiện hình dạng và đơn giản hóa cấu trúc của các đối tượng.

Các phép toán này sử dụng một phần tử cấu trúc được gọi là nhân (kernel), ma trận này thường là một ma trận vuông, ma trận này sẽ trượt qua ảnh, thực hiện phép toán logic tập hợp trên các pixel lân cận

b) Vì sao cần xử lý hình thái học

Một trong những vấn đề khá phổ biến trong xử lý ảnh số ta có thể nêu đến như nhiễu (các chấm trắng li ti) hoặc lỗ thủng (các vết đen), và các phép toán trong xử lý hình thái học có thể giúp ta xử lý các vấn đề trên.

Mask thu được từ việc lấy mask HSV có thể chứa nhiễu hoặc lỗ hổng trong các vùng liên quan tới biển báo. Morphology có thể giúp giải quyết các việc trên.

1.3.1 Phép co (erode) và Phép giãn (dilate)

1.3.1.1 Phép co (erode)

Giống với tên gọi, phép co sẽ làm các vùng trắng co lại và làm giãn nở các vùng đen.

Tại mỗi pixel trung tâm, chỉ khi tất cả các pixel nằm dưới kernel đều là trắng (255) thì pixel mới được giữ lại là trắng. Nếu có bất kỳ pixel đen nào, pixel trung tâm

sẽ trở thành đen. (Có thể xem như tương đương với phép toán logic AND trên tập điểm ảnh)

Điều này sẽ rất hiệu quả trong việc xóa các chấm nhỏ và làm mỏng các đường biên. Các chấm nhiễu nhỏ sẽ biến mất hoàn toàn sau một hoặc vài lần sử dụng phép co.

1.3.1.2 Phép giãn (dilate)

Ngược lại với phép co, phép giãn sẽ làm giãn các vùng trắng (255) và làm co lại các vùng đen (0).

Tại mỗi pixel trung tâm, nếu có ít nhất một pixel nằm dưới kernel là trắng (255) thì pixel trung tâm sẽ trở thành trắng. (Có thể xem tương đương với phép toán logic OR trên tập điểm ảnh).

Điều này sẽ rất hiệu quả trong việc làm liền các vết nứt, khe hở, hoặc lấp đầy các lỗ thủng nhỏ nằm bên trong đối tượng.

1.3.2 Phép mở (opening) và phép đóng (closing)

Đây là các phép toán tổng hợp từ 2 phép co và phép giãn ở phần trước, nó được dùng để đạt mục đích xử lý chính xác hơn

Table 1. Phép mở và Phép đóng

Phép toán	Cấu trúc	Mục đích
Phép mở	Erode => Dilate	Phép mở cho phép loại bỏ nhiễu một cách hiệu quả. Ban đầu sẽ dùng phép co để xóa sạch nhiễu của vật thể, sau đó sẽ dùng phép giãn để khôi phục kích thước vật thể, đảm bảo không bị biến dạng hoặc teo nhỏ.
Phép đóng	Dilate => Erode	Phép đóng cho phép lấp đầy lỗ thủng bên trong đối tượng. Đầu tiên dùng phép giãn để lấp lỗ, sau đó dùng phép co để làm mượt lại đường biên ngoài cùng.

1.4 Tìm viền (Finding Countour)

a) Định nghĩa

Viền chính là một đường cong khép kín nối các điểm biên của đối tượng, nơi cường độ pixel thay đổi từ nhiều nhất từ tiền cảnh (foreground) sang hậu cảnh (background).

Trong ảnh nhị phân với tiền cảnh = 255 và hậu cảnh bằng = 0, thì viền chính là những đường bao ngoài cùng của các vùng trắng liên thông với nhau.

Viền cũng có thể biểu diễn dưới dạng vector của các điểm tọa độ (x,y)

⇒ Từ đây, có thể dùng viền để suy ra được các tính chất như

- Diện tích
- Chu vi
- Hình dạng của vật thể

b) Mục đích của việc sử dụng viền

Như đã phân tích, việc tìm viền của vật thể có thể giúp ta phân tích được các đặc trưng sâu hơn của chính vật thể đó. Nói cách khác viền đóng vai trò là bước chuyển đổi thiết yếu từ dữ liệu pixel sang đối tượng hình học có thể đo lường và phân tích được.

Sau khi tính toán xong mặt nạ nhị phân và đã làm sạch bằng các phép toán xử lý hình thái học, ta cần xác định viền để tính toán các tham số định lượng như diện tích, chu vi, và một giá trị đặc biệt là độ tròn Circularity (mục 1.5). Không có viền, ta sẽ không thể tính đánh giá được chính xác hình dạng của một đối tượng nào, từ đó có thể bỏ qua các đối tượng mà ta muốn lấy.

Các viền còn giúp ta lấy được ROI, hay vùng quan tâm, vì viền cung cấp một tập hợp tọa độ chính xác của đường bao đối tượng. Dựa trên tập hợp này ta dễ dàng tính toán được hình chữ nhật bao ngoài vật thể, và cũng nhờ hình chữ nhật bao ngoài này ta có thể cắt chính xác được ROI của vật thể, phục vụ cho các kỹ thuật so sánh nâng cao.

Ngoài ra, nhờ việc sử dụng các chỉ số từ các đường viền (diện tích, độ tròn) ta còn có thể dùng như một lớp lọc nâng cao hơn để loại bỏ các đối tượng không phù

hợp ngay từ sớm, ta giảm đáng kể lượng vùng ảnh cần đưa vào các bước nhận diện phức tạp. Điều này giúp tối ưu hóa hiệu năng.

c) Cách áp dụng

Việc tìm đường viền trong OpenCV thường được triển khai bằng hàm `findContours`, ta sẽ chỉ tập trung vào 2 tham số quan trọng nhất được sử dụng trong bài báo cáo:

Table 2. Thuộc tính của `findContours`

Tham số	Giá trị áp dụng	Logic xử lý
Retrieval Mode (Giá trị truy xuất)	<code>cv2.RETR_EXTERNAL</code>	Giảm tính toán và giảm nhiễu. Ta chỉ quan tâm đến đường bao ngoài cùng của các đối tượng. Việc này bỏ qua các Contour lồng bên trong (ví dụ: các chi tiết hình vẽ bên trong biển báo hoặc các lỗ thủng nhỏ còn sót)
Approximation Method (Phương pháp xấp xỉ)	<code>cv2.CHAIN_APPROX_METHOD</code>	Tối ưu bộ nhớ và tốc độ xử lý. Thuật toán loại bỏ các điểm thừa nằm trên đường thẳng. Ví dụ, thay vì lưu hàng trăm tọa độ cho một cạnh thẳng của hình chữ nhật, nó chỉ lưu 4 đỉnh đặc trưng (corner points).

Nhờ lựa chọn này mà đường bao được mô tả đầy đủ và các tính toán đặc trưng hình dạng về sau tăng độ chính xác.

1.5 Phân tích độ tròn

a) Định nghĩa

Hàm phân tích độ tròn được ứng dụng trong bài toán là một đại lượng hình học được dùng để đo mức độ “giống hình tròn” của một đối tượng.

Công thức được sử dụng như sau:

$$Circularity = \frac{4\pi * \text{Diện tích}}{(\text{Chu vi})^2}$$

Với:

- Diện tích là diện tích của viền
- Chu vi là chu vi của viền có được từ đường bao

Ý nghĩa hình học:

- Nếu chu vi nhỏ tương ứng với diện tích lớn → hình dạng tròn và gọn biên
- Nếu chu vi lớn nhưng diện tích nhỏ → hình dạng dẹt, méo, hoặc có nhiều cạnh sắc

b) Mục đích sử dụng

Mặc dù được gọi là hàm “đo độ tròn”, nhưng Circularity được sử dụng như một bộ lọc hình dạng tổng quát để phân tách các đối tượng phù hợp với bộ biển báo cần nhận dạng.

Vì đối với hình tròn chuẩn, vùng Circularity có thể dao động từ 0.7 đến đúng 1.0, còn với hình tam giác thì vùng có thể dao động từ 0.5 đến 0.7.

Việc sử dụng hàm này như một hàm nhận dạng có thể giúp tìm được các đối tượng mà ta mong muốn trong bài toán mà đối với bài toán nhận diện biển báo là 2 hình tròn và tam giác.

c) Cách hoạt động

Đầu tiên ta sẽ tính diện tích và chu vi của viền bằng các hàm được xây dựng sẵn trong OpenCV, rồi áp dụng vào công thức circularity mà ta tự định nghĩa.

Đối với các viền có circularity > 0.7, ta sẽ nhận là hình tròn, còn các viền có circularity > 0.5, ta sẽ phải xét thêm điều kiện, tránh nhầm lẫn với các vật thể nhiễu.

Lưu ý rằng ngưỡng trên là bộ lọc được tinh chỉnh theo nhiều lần thực nghiệm để phù hợp với video và ánh sáng thực tế.

1.6 Trích xuất vùng quan tâm (ROI)

a) Định nghĩa

Vùng trung tâm hay thường được gọi là ROI là một vùng ảnh con chứa đối tượng cần được phân tích sâu hơn, trong bài toán này là biển báo giao thông.

Việc trích xuất vùng quan tâm giúp ta trong việc xử lý rất nhiều, bởi vì thay vì phải xử lý toàn bộ video, ta chỉ cần làm việc với những vùng có khả năng là biển báo.

ROI thường được xác định bằng cách dựa vào hình chữ nhật bao quanh đối tượng.

$$ROI = frame[y : y + h, x : x + w]$$

Trong đó (x,y) là cặp tọa độ góc trên bên trái của vùng được bao bởi viền.

b) Mục đích sử dụng

ROI đóng vai trò nền tảng để chuyển từ bước “phát hiện” sang bước “nhận dạng”.

Việc dùng ROI cho các lợi ích sau:

- Giảm độ phức tạp tính toán: Thay vì phải chạy thuật toán khớp đặc trưng (Mục 1.7) trên toàn bộ frame ảnh, ta chỉ cần tập trung xử lý trên một khối không gian nhỏ hơn nhiều, đó chính là vùng ảnh chứa biển báo đã được cắt. Điều này giảm thiểu đáng kể chi phí tính toán, giúp hệ thống đạt được tốc độ xử lý tốt hơn.
- Tăng độ chính xác nhận dạng: việc so sánh đặc trưng được thực hiện trên vùng ảnh đã được cắt sạch giúp tăng cường độ tương đồng giữa ROI và template
- Tập trung vào đối tượng quan trọng: ROI giúp loại bỏ các nền gây nhiễu khác trên frame.

c) Cách hoạt động

Sau khi tìm được các đường viền có tính chất phù hợp từ bước Circularity, hệ thống sẽ lựa chọn các contour này làm ứng viên biển báo và tiến hành trích xuất thông

tin hình học từ chúng. Cụ thể, đối với mỗi contour, ta tính toán hình chữ nhật bao ngoài (Bounding Box) bằng hàm `cv2.boundingRect()`; Hình chữ nhật này cung cấp tọa độ góc trên bên trái cùng với chiều rộng và chiều cao của vùng ảnh nơi biển báo xuất hiện.

Dựa trên các tọa độ này, ta tiến hành cắt ra vùng ảnh con từ **ảnh gốc màu** (không phải từ mặt nạ nhị phân), bởi việc nhận dạng nội dung bên trong biển báo (như mũi tên, dấu hiệu, con số) đòi hỏi phải giữ nguyên thông tin hình ảnh ở phần nội dung, đặc biệt là các họa tiết hoặc màu sắc đặc trưng của từng loại biển báo. Để tránh hiện tượng cắt cụt viền hay mất thông tin quan trọng ở sát biên, một khoảng đệm (padding) nhỏ được cộng thêm vào bounding box trước khi cắt ROI.

Vùng ROI sau đó được chuẩn hóa kích thước (ví dụ resize về 100×100) nhằm đảm bảo tính đồng nhất giữa tất cả mẫu đầu vào của bước matching, vì cơ chế so khớp đặc trưng yêu cầu sự tương thích về kích thước giữa các ảnh so sánh.

1.7 Nhận diện biển báo bằng ORB Feature Matching

1.7.1 Trích xuất và mô tả đặc trưng bằng ORB

a) Đặc trưng và ORB

- Đặc trưng là các điểm ảnh (keypoints) hoặc vùng ảnh nhỏ có tính chất độc nhất và dễ nhận biết trong một bức ảnh. Đặc trưng đóng vai trò như “dấu vân tay” của đối tượng.
- ORB (Oriented FAST and Rotated BRIEF): Là một thuật toán được tối ưu hóa về tốc độ để trích xuất và mô tả đặc trưng thuộc thư viện OpenCV

b) Mục đích sử dụng

ORB cho phép hệ thống nhận diện biển báo ngay cả khi chúng bị xoay hoặc thay đổi góc nhìn. Đây là lí do chính khiến chúng em chọn Feature Matching thay vì Template Matching, việc này sẽ giúp nhận diện được các biển báo bị nghiêng.

ORB không chỉ tìm ra điểm đặc trưng (keypoints) mà còn tạo ra một vector mô tả (descriptor) nhị phân cho mỗi điểm đó, giúp máy tính so sánh tính chất của chúng một cách hiệu quả

1.7.2 Nhận dạng biển báo bằng Feature Matching sử dụng ORB

a) Feature Matching

Ta có thể hiểu Feature Matching là quá trình so sánh các vector mô tả đặc trưng được trích xuất từ ROI so với các vector mô tả đã được lưu trữ sẵn (từ ảnh mẫu – template)

Mục tiêu của ta là xác định mẫu nào trong thư viện có số lượng đặc trưng trùng khớp cao nhất với ROI

b) Cách hoạt động

- Phân loại (Classification): Keypoints và Descriptors chỉ nói cho ta biết những điểm độc đáo của biển báo. Matching là bước cuối cùng để gán nhãn, xác định loại biển báo đó là gì (Tốc độ 60, Cấm rẽ trái, v.v.).
- Tìm sự tương đồng (Similarity): Feature Matching giúp tìm ra sự tương đồng về cấu trúc và hình ảnh giữa ROI và Template, thay vì chỉ so sánh màu sắc hay pixel thô.

c) Luồng hoạt động

Quá trình nhận dạng bằng ORB Feature Matching được triển khai ngay sau khi đã có ROI từ bước trích xuất vùng quan tâm. Đầu tiên, ROI được chuẩn hoá về mặt kích thước và ánh sáng nhằm đảm bảo tính tương thích so sánh với các Template trong thư viện. Sau khi chuẩn hoá, thuật toán ORB sẽ tiến hành phát hiện các điểm đặc trưng (keypoints) và xây dựng các vector mô tả đặc trưng (descriptors) cho ROI.

Ở phía còn lại, bộ Template của mỗi loại biển báo cũng đã được xử lý tương tự trước đó: tất cả các hình mẫu đều được resize, cân bằng sáng tối và lưu trữ sẵn danh sách keypoints cùng descriptors. Khi bắt đầu so khớp, hệ thống sẽ duyệt qua từng loại biển báo trong thư viện và so sánh descriptors của ROI với descriptors của từng Template bằng bộ so khớp BFMatcher sử dụng độ đo khoảng cách Hamming.

Các cặp đặc trưng có mức tương đồng cao (good matches) sẽ được giữ lại, từ đó tính toán ra một mức điểm Similarity Score thể hiện mức độ phù hợp giữa ROI và Template. Nếu điểm số vượt ngưỡng nhận dạng (threshold), biển báo sẽ được gán nhãn với loại Template tương ứng. Trong trường hợp điểm số thấp hơn ngưỡng cho phép, đối tượng được xem như không xác định và gán nhãn “Unknown”.

Như vậy, Feature Matching chính là cầu nối cuối cùng giúp hệ thống chuyển từ việc phát hiện biển báo sang hiểu nội dung của biển báo. Đây là thành phần cốt lõi để hệ thống phân biệt rõ ràng giữa các biển báo có hình dạng giống nhau nhưng ý nghĩa hoàn toàn khác nhau (ví dụ: Cấm rẽ trái và Cấm rẽ phải).

d) Tóm tắt và Kết luận

Nhận diện biển báo bằng ORB Feature Matching là bước cuối cùng và là cơ chế suy luận cốt lõi của toàn bộ hệ thống. Bước này đóng vai trò quyết định, chuyển hệ thống từ việc chỉ Phát hiện một đối tượng (sau khi lọc màu và tìm viền) sang việc hiểu rõ nội dung của đối tượng đó.

Feature Matching chính là thành phần cốt lõi giúp hệ thống phân biệt rõ ràng giữa các biển báo có hình dạng giống nhau nhưng ý nghĩa khác nhau (ví dụ: Cấm rẽ trái và Cấm rẽ phải).

1.8 Kết luận

Chương 1 đã trình bày chi tiết và có hệ thống toàn bộ chuỗi xử lý ảnh số (DIP Pipeline) được áp dụng để giải quyết bài toán Nhận dạng Biển báo Giao thông. Các phương pháp được lựa chọn không chỉ dựa trên nguyên tắc cơ bản của môn học mà còn dựa trên tư duy tối ưu hóa và phản biện nhằm giải quyết các thách thức thực tế.

Cụ thể, hệ thống được xây dựng trên một nền tảng logic vững chắc: Lọc màu HSV được ưu tiên để đảm bảo tính ổn định trước sự thay đổi của ánh sáng, khắc phục nhược điểm của không gian RGB. Tiếp theo, các phép toán Morphology và Phân tích Contours/Circularity đảm bảo rằng các đối tượng được đưa vào bước nhận dạng là những ứng viên có hình dạng chuẩn và đã được làm sạch nhiễu.

Cuối cùng, việc lựa chọn ORB Feature Matching làm cơ chế suy luận là quyết định chiến lược, mang lại tính bất biến cho hệ thống, cho phép nhận dạng thành công các biển báo bị xoay hoặc nhìn từ góc nghiêng.

Sự kết hợp logic giữa các kỹ thuật Phân đoạn (HSV), Hình học (Contours) và Nhận dạng (ORB) này là yếu tố cốt lõi, đảm bảo độ chính xác và độ tin cậy của toàn bộ đồ án.

CHƯƠNG 2. THỰC NGHIỆM VÀ KẾT QUẢ

2.1 Import và hàm circularity

```
import cv2
import numpy as np
import os
import glob

def circularity(cnt):
    """
    Tính toán độ tròn (circularity). 1.0 là hình tròn hoàn hảo.
    """
    a = cv2.contourArea(cnt)
    p = cv2.arcLength(cnt, True)
    if a < 1e-6 or p < 1e-6: return 0.0
    return float(4 * np.pi * a / (p * p))
```

Figure 1. Import & Circularity

Import các thư viện cần thiết cho bài toán, chúng ta có:

- Thư viện OpenCV để thực hiện các thao tác xử lý ảnh.
- Thư viện numpy để thực hiện tính toán.
- Thư viện os để quản lý thư mục “template”.
- Thư viện glob dùng để tìm các file có dạng “*.jpg, *.png”.

Tiếp theo là hàm đánh giá “độ tròn” của đối tượng, hàm này nhận vào một viền và tính toán 2 thông số hình học:

- a: Diện tích của viền.
- p: Chu vi của viền.

Với công thức được nêu ra ở chương 1, hàm này trả về tham số độ tròn của vật thể được truyền vào, là một hàm hỗ trợ cho các logic phía dưới.

2.2 Load các template và trích xuất đặc trưng ORB

```

15 def load_templates(template_dir="template"):
16     """
17     Load template
18     """
19     templates_orb = {}
20     orb = cv2.ORB_create(nfeatures=1000)
21
22     if not os.path.exists(template_dir):
23         print(f"Warning: Template directory '{template_dir}' not found!")
24         return templates_orb
25
26     for label_folder in os.listdir(template_dir):
27         folder_path = os.path.join(template_dir, label_folder)
28         if not os.path.isdir(folder_path):
29             continue
30
31         label = label_folder
32         image_files = []
33         for ext in ("*.jpg", "*.png", "*.jpeg"):
34             image_files.extend(glob.glob(os.path.join(folder_path, ext)))
35
36         if len(image_files) == 0:
37             print(f"Warning: Folder '{label}' does not contain any images.")
38             continue
39
40         kp_list = []
41         des_list = []
42         img_list = []
43
44         for img_path in image_files:
45             img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
46             if img is None:
47                 print(f"Warning: Cannot read {img_path}")
48                 continue
49
50             img = cv2.resize(img, (100, 100))
51             img = cv2.equalizeHist(img)
52
53             kp, des = orb.detectAndCompute(img, None)
54             if des is not None:
55                 kp_list.append(kp)
56                 des_list.append(des)
57                 img_list.append(img)
58         if len(des_list) == 0:
59             print(f"Warning: No features found in any images of '{label}'")
60             continue
61         templates_orb[label] = {
62             "imgs": img_list,
63             "kp": kp_list,
64             "des": des_list,
65             "label": label
66         }
67         print(f"Loaded template: {label} - {sum(len(k) for k in kp_list)} total keypoints ({len(kp_list)} images)")
68     print(f"Total templates loaded: {len(templates_orb)}")
69     return templates_orb

```

Figure 2. Load và trích xuất đặc trưng của template

Chức năng chính: Đọc template từ thư mục -> trích xuất keypoints + descriptor bằng ORB -> lưu dưới dạng cấu trúc để sử dụng trong bước nhận dạng

Ta sẽ đi sâu vào từng phân đoạn trong hàm trên.

2.2.1 Tạo cấu trúc lưu dữ liệu + ORB detector

```
templates_orb = {}
orb = cv2.ORB_create(nfeatures=1000)
```

- Tạo từ điển templates_orb để lưu template theo từng biển báo, giúp truy xuất nhanh khi phân loại.
- Tạo ORB với 1000 đặc trưng tối đa, giúp giảm miss match với template có độ chi tiết thấp.

2.2.2 Kiểm tra thư mục chứa templates

```
if not os.path.exists(template_dir):
    print(f"Warning: Template directory '{template_dir}' not found!")
    return templates_orb
```

- Kiểm tra thư mục, nếu không có thư mục thì trả lỗi, tránh gây crash chương trình.

2.2.3 Duyệt từng thư mục biển báo

```
for label_folder in os.listdir(template_dir):
    folder_path = os.path.join(template_dir, label_folder)
    if not os.path.isdir(folder_path):
        continue
```

- Duyệt qua từng folder, mỗi folder con là một loại biển báo

2.2.4 Lấy danh sách file ảnh trong từng loại

```
image_files = []
for ext in ("*.jpg", "*.png", "*.jpeg"):
    image_files.extend(glob.glob(os.path.join(folder_path, ext)))
```

- Lấy danh sách file ảnh trong từng loại, chấp nhận nhiều định dạng ảnh, có báo lỗi nếu thư mục rỗng

2.2.5 Trích xuất đặc trưng template

```

for img_path in image_files:
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        print(f"Warning: Cannot read {img_path}")
        continue

    img = cv2.resize(img, (100, 100))
    img = cv2.equalizeHist(img)

    kp, des = orb.detectAndCompute(img, None)

```

- Vòng lặp qua từng `img_path` để trích xuất các đặc trưng từ ảnh template.
- Đầu tiên là chuyển về ảnh xám vì ORB hoạt động trên ảnh xám.
- Ta chuyển kích thước ảnh thành 100x100 để tăng độ tương thích của ROI so với Template.
- Sử dụng histogram equalization để làm rõ chi tiết, nhằm tăng keypoint.
- Dùng hàm `detectAndCompute` để lấy ra keypoints và descriptor dùng trong matching.

2.2.6 Lưu và trả về sau khi đã chạy qua hết

```

        continue
    templates_orb[label] = {
        "imgs": img_list,
        "kp": kp_list,
        "des": des_list,
        "label": label
    }

```

- Ta tiến hành lưu lại các giá trị rồi trả về output.

2.2.7 Kết luận khối code

Khối code này chủ yếu để xử lý những gì liên quan tới template, là một hàm hỗ trợ lấy và trích xuất đặc trưng từ template.

Các template được lấy từ video và qua tìm kiếm trên mạng nhằm mục đích đạt xác suất cao nhất trong việc so khớp mẫu.

2.3 So khớp đặc trưng

Khối code này chủ yếu được sử dụng trong việc so khớp đặc trưng giữa ROI và các template để quyết định nhãn của biển báo.

```

71 def orb_classify(roi_bgr, templates_orb, orb_detector, bf_matcher):
72     """
73     Phân loại ROI bằng ORB Feature Matching.
74     """
75     if roi_bgr is None or roi_bgr.size == 0:
76         return "Unknown", 0.0
77
78     # chuyển đổi ROI sang ảnh xám
79     roi_gray = cv2.cvtColor(roi_bgr, cv2.COLOR_BGR2GRAY)
80     roi_gray = cv2.resize(roi_gray, (100, 100))
81     roi_gray = cv2.equalizeHist(roi_gray)
82
83     # Tìm keypoint + descriptor của ROI
84     kp_roi, des_roi = orb_detector.detectAndCompute(roi_gray, None)
85
86     # Nếu không đủ đặc trưng thì bỏ
87     if des_roi is None or len(des_roi) < 1:
88         return "Unknown", 0.0
89
90     best_label = "Unknown"
91     best_score = 0.0
92     threshold = 0.30
93
94     # ---- Lặp qua từng template để so khớp ----
95     for label, tpl in templates_orb.items():
96         des_list = tpl.get('des', [])
97         kp_list = tpl.get('kp', [])
98         label_best = 0.0
99         for des_tpl, kp_tpl in zip(des_list, kp_list):
100             try:
101                 matches = bf_matcher.match(des_tpl, des_roi)
102                 good_matches = [m for m in matches if m.distance < 70]
103                 score = len(good_matches) / (len(kp_tpl) + 1e-6)
104                 if score > label_best:
105                     label_best = score
106             except Exception:
107                 continue
108         if label_best > best_score:
109             best_score = label_best
110             best_label = label
111
112     # Không đủ match thì đặt label là Unknown
113     if best_score < threshold:
114         return "Unknown", best_score
115
116     return best_label, best_score
117

```

Figure 3. Orb Classify

2.3.1 Kiểm tra đầu vào

```
if roi_bgr is None or roi_bgr.size == 0:
    return "Unknown", 0.0
```

- Nếu ROI có lỗi hoặc rỗng, bỏ qua và trả về để tránh gây crash chương trình

2.3.2 Chuẩn hóa ROI

```
# chuyển đổi ROI sang ảnh xám
roi_gray = cv2.cvtColor(roi_bgr, cv2.COLOR_BGR2GRAY)
roi_gray = cv2.resize(roi_gray, (100, 100))
roi_gray = cv2.equalizeHist(roi_gray)
```

- Ta chuyển ROI về ảnh xám vì ORB hoạt động với ảnh xám.
- Resize về 100x100 để đồng bộ với template.
- Cân bằng histogram để tăng độ tương phản, có thể giúp tăng keypoints.

2.3.3 Trích xuất đặc trưng ROI

```
# Tìm keypoint + descriptor của ROI
kp_roi, des_roi = orb_detector.detectAndCompute(roi_gray, None)

# Nếu không đủ đặc trưng thì bỏ
if des_roi is None or len(des_roi) < 1:
    return "Unknown", 0.0
```

- Nếu ROI không có keypoints và descriptor thì bỏ vì không thể nhận dạng, tránh nhận bừa các điểm nhiễu.

2.3.4 Matching với template

```
for label, tpl in templates_orb.items():
    des_list = tpl.get('des', [])
    kp_list = tpl.get('kp', [])
    label_best = 0.0
```

- Lặp qua từng loại biển báo, một loại biển báo có thể có nhiều ảnh template. Ta sẽ test ROI với từng ảnh template trong loại đó, lấy kết quả tốt nhất làm đại diện cho cả label

```

for des_tpl, kp_tpl in zip(des_list, kp_list):
    try:
        matches = bf_matcher.match(des_tpl, des_roi)
        good_matches = [m for m in matches if m.distance < 70]
        score = len(good_matches) / (len(kp_tpl) + 1e-6)
        if score > label_best:
            label_best = score
    except Exception:
        continue

```

- Lặp qua từng ảnh template trong cùng 1 loại biển báo
 - Des_list là descriptor của ảnh template thứ i, kp_list là danh sách keypoints tương ứng, ta dùng zip để đảm bảo đúng cặp des kp.
- matches = bf_matcher.match(des_tpl, des_roi)
 - Dùng brute-force matcher với hamming distance để so từng descriptor của template với descriptor của ROI.
 - Output là một list matches, mỗi phần tử chứa một cặp (idx_template, idx_roi, distance)
 - good_matches = [m for m in matches if m.distance < 70]
 - distance càng nhỏ thì descriptor càng giống nhau.
 - Điều kiện <70 là một ngưỡng thực nghiệm, loại bớt những match yếu.
 - score = len(good_matches) / (len(kp_tpl) + 1e-6)
 - Tử số: số lượng match tốt.
 - Mẫu số: tổng kp của ảnh template đó.
 - Thêm +1e-6 để tránh chia cho 0, gây crash chương trình.
 - if score > label_best:
 - label_best = score
 - Giữ lại score cao nhất trong tất cả template ảnh của cùng một loại biển
 - Cùng một loại biển nhưng ảnh chụp khác góc, khác ánh sáng, có thể sẽ đưa ra output khác nhau, nên phải check.

```

        continue
    if label_best > best_score:
        best_score = label_best
        best_label = label

```

- Chọn label theo best_score
- Nếu không đủ match thì đặt label là unknow
- Hàm trả về best_label và best_score

2.3.5 Kết luận

Với mỗi ROI, hệ thống so khớp nó với toàn bộ ảnh mẫu của từng loại biển báo, tính tỷ lệ đặc trưng trùng khớp, chọn loại có tỷ lệ cao nhất, và chỉ chấp nhận gán nhãn nếu tỷ lệ đó vượt qua một ngưỡng tin cậy tối thiểu.

2.4 Hàm xử lý chính

2.4.1 Khởi tạo


```

def main():
    # Khởi tạo Video I/O
    cap = cv2.VideoCapture('input_videos/video2.mp4')
    if not cap.isOpened():
        print("Error opening video file")
        exit()

    frame_width = int(cap.get(3))
    frame_height = int(cap.get(4))
    size = (frame_width, frame_height)
    fps = cap.get(cv2.CAP_PROP_FPS) or 30.0

    result = cv2.VideoWriter('video2_output.mp4',
                             cv2.VideoWriter_fourcc(*'mp4v'),
                             fps, size)

    # Khởi tạo ORB và Matcher
    orb_detector = cv2.ORB_create(nfeatures=1500)
    bf_matcher = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

    # Load Templates
    templates_orb = load_templates("template")

    use_feature_matching = len(templates_orb) > 0

    # Dải màu HSV
    color_ranges = {
        "Red": [
            (np.array([0, 40, 20]), np.array([12, 255, 255])),
            (np.array([160, 40, 20]), np.array([179, 255, 255]))
        ],
        "Blue": [(np.array([100, 150, 0]), np.array([140, 255, 255]))],
    }

    # Kernels
    k5 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    k7 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))

```

Figure 4. Init trong hàm main

Ta khởi tạo các biến cần thiết cho việc xử lý video và các biến hỗ trợ trong việc tính toán và nhận dạng.

Table 3. Các biến khởi tạo

Tên biến	Ý nghĩa	Vai trò
----------	---------	---------

cap	Biến dùng để đọc video đầu vào	Lấy từng frame để nhận dạng
frame-width frame_height	Kích thước khung hình	Dùng để set up video
fps	Số frame trên giây của video	Xác định tốc độ chuyển động của video
result	Dùng để ghi video đầu ra	Lưu video đầu ra
orb_detector	Bộ trích xuất đặc trưng orb với 3000 keypoints	Tăng độ chính xác khi nhận dạng ROI
bf_matcher	Bộ so khớp descriptor theo kiểu brute_force	Tìm điểm tương đồng giữa ROI và template
templates_orb	Thư viện ảnh template đã xử lý đặc trưng ORB	Cơ sở dữ liệu để so khớp nhận dạng biển báo
use_feature_matching	Kiểm tra liệu template có được load thành công không	Chỉ kích hoạt ORB nếu dữ liệu có sẵn
color_ranges	Ngưỡng HSV cho màu đỏ & xanh biển báo	Phân tách vùng chứa biển báo khỏi background
k5, k7	Kernel morphology dạng ellipse	Giảm nhiễu & làm rõ vùng biển báo trên mask

2.4.2 Xử lý hình thái học

```

"""Xử lý màu"""
for color, ranges in color_ranges.items():
    mask = np.zeros(hsv.shape[:2], dtype=np.uint8)
    for (lo, hi) in ranges:
        mask |= cv2.inRange(hsv, lo, hi)

```

Figure 5. Xử lý màu

- Ta sẽ duyệt qua từng màu trong danh sách để xử lý riêng, vì mỗi màu có đặc trưng riêng

```

"""
Morphology
Phần xử lý được xử lý riêng màu đỏ và màu xanh vì các đặc trưng riêng
"""
if color == "Blue":
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k5, 1)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (16, 16)), 2)
    mask = cv2.dilate(mask, k5, iterations=1)
elif color == "Red": #đỏ
    mask = cv2.dilate(mask, k7, 2)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k7, 2)

```

Figure 6. Xử lý hình thái học

- Đối với màu xanh dương, ta bắt đầu open trước để giảm nhiễu.
- Sau đó closing để làm kín các lỗ hổng trong biển báo, ở đây, ta chọn closing lớn vì các biển xanh thường đặc ruột, dễ bị đứt nét khi threshold.
- Ta tiếp tục dilate thêm lần nữa để chắc chắn việc làm liền nét đứt của biển báo.
- Đối với màu đỏ, vì màu đỏ thường rỗng ruột nên ta sẽ dilate trước để làm đầy, rồi tiếp theo là close để giữ vùng viền nguyên vẹn, tránh bị đứt nét.
- Khối mã xử lý màu dựa trên không gian HSV để phân đoạn các vùng có khả năng là biển báo giao thông. Hai nhóm biển báo có đặc trưng màu sắc khác nhau được xử lý riêng: biển xanh được làm mượt và khôi phục cấu trúc vùng đặc, trong khi biển đỏ được nhấn mạnh viền vòng ngoài. Việc áp dụng riêng các phép morphology giúp giảm nhiễu mà

vẫn bảo toàn hình dạng đặc trưng của từng loại biển báo, làm tiền đề cho tiến trình phát hiện contour và nhận dạng nội dung ở các bước sau.

2.4.3 Loại nhiễu

```
# Tìm contour trên mask
cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

if not cnts:
    continue

for c in cnts:
    area = cv2.contourArea(c)
    if area < 1500:
        continue

    # Lọc Aspect Ratio
    x, y, w_rect, h_rect = cv2.boundingRect(c)
    ar = w_rect / float(h_rect)
    if ar < 0.6 or ar > 1.5:
        continue

    circ = circularity(c)
```

Figure 7. Loại nhiễu

- Ta thực hiện việc tìm viền và sau đó là xét, nếu một viền xuất hiện nhưng quá nhỏ => xem đó là nhiễu và bỏ qua, không xét.
- Tiếp theo ta lọc bằng aspect ratio (tỉ lệ chiều rộng / chiều cao), điều kiện là với các ar nhỏ, thì hình dạng thường là dẹt theo chiều cao (dạng cột), biển báo sẽ thường không có dạng này nên ta loại bỏ, với các ar quá lớn (dẹt và dài ngang) ta cũng loại bỏ vì không hợp với đề bài.
- Hình tròn và tam giác vẫn được chấp nhận trong khoảng còn lại.
- Ta áp dụng hàm circularity đã trình bày ở trên để tính “độ tròn” của một vật thể

```

if color == "Red":
    if circ > 0.70:
        roi = mask[y:y + h_rect, x:x + w_rect]
        fill = roi.mean() / 255.0
        if fill > 0.01:
            detections.append((x, y, w_rect, h_rect, color))

    elif circ > 0.50:
        roi = mask[y:y + h_rect, x:x + w_rect]
        fill = roi.mean() / 255.0
        # Bỏ vật thể đỏ đặc (fill cao) != hình tam giác
        if fill > 0.50:
            continue

        # Giữ biển báo đỏ: viền đỏ, ruột vàng nên fill vừa phải
        if 0.05 < fill < 0.55:
            detections.append((x, y, w_rect, h_rect, color))

elif color == "Blue":
    # chỉ kiểm tra hình tròn cho biển xanh
    if circ > 0.70:
        roi = mask[y:y + h_rect, x:x + w_rect]
        fill = roi.mean() / 255.0
        if fill > 0.35:
            detections.append((x, y, w_rect, h_rect, color))

```

Figure 8. Xử lý theo màu

- Ở đây, fill đại diện cho độ rộng ruột của vật thể, nếu fill càng tiệm cận 1 thì càng đặc, càng tiệm cận 0 thì càng rỗng
- Đối với màu đỏ, ta sẽ xét 2 trường, đầu tiên là xét nếu màu đỏ là một hình tròn ($\text{circ} > 0.70$).
- Đối với hình tròn, sẽ có $\text{circ} > 0.70$ và $\text{fill} > 0.01$, có viền đỏ và phần ruột không bị quá rỗng.
- Đối với tam giác circ sẽ từ 0.50 đến 0.70, với điều kiện là fill phải từ 0.05 đến 0.55, không quá rỗng và không quá đặc.
- Đối với màu xanh, ta chỉ cần check $\text{circ} > 0.70$ và $\text{fill} > 0.35$.

2.5 Xử lý feature matching

```

recognized_signs = []
if use_feature_matching:
    for (x, y, w_rect, h_rect, color) in detections:
        # Cắt ROI từ frame gốc (thêm padding nhỏ)
        padding = 5
        x1 = max(0, x - padding)
        y1 = max(0, y - padding)
        x2 = min(frame_width, x + w_rect + padding)
        y2 = min(frame_height, y + h_rect + padding)

        roi_bgr = frame[y1:y2, x1:x2]

        if roi_bgr.size == 0:
            continue

        # Phân loại bằng feature matching
        label, score = orb_classify(roi_bgr, templates_orb, orb_detector, bf_matcher)

        if label != "Unknown":
            recognized_signs.append((x, y, w_rect, h_rect, color, label, score))
else:
    pass

```

Figure 9. Xử lý feature matching

- Đầu tiên ta khởi tạo một danh sách kết quả các biển báo được nhận diện thành công (recognized_signs)
- Nếu đã load đủ template thành công, ta bắt đầu thực hiện vòng lặp xử lý feature matching
- Ta lặp qua từng detection của bước trước đó, mỗi contour khả nghi sẽ đi vào bước nhận diện.
- Bắt đầu tính x1,y1,x2,y2 và padding để cắt ROI từ frame gốc vì ROI từ mask chỉ giữ màu trắng đen, nhận diện ký hiệu cần màu thật của biển báo.
- Ta tiến đến gọi hàm so khớp đặc trưng orb_classify để lấy nhãn và điểm tin cậy.
- Ta append vào list kết quả các biển báo với nhãn khác với Unknown.

2.6 Vẽ output

```

# Biển báo bị phát hiện nhưng không được nhận dạng
for (x, y, w, h, color) in detections:
    if (x, y, w, h) not in recognized_positions:
        cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 255, 0), 3)
        cv2.putText(vis, "", (x, y - 5),
                     cv2.FONT_HERSHEY_SIMPLEX, 0.8,
                     (0, 255, 255), 2, cv2.LINE_AA)
# Vẽ đối tượng đã nhận dạng được
for (x, y, w, h, color, label, score) in recognized_signs:
    cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 255, 0), 3) # xanh lá
    display_label = label
    cv2.putText(vis, f"{display_label}", (x, y - 10),
                 cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2, cv2.LINE_AA)

result.write(vis)
cv2.imshow('EndTerm Output', vis)

key = cv2.waitKey(1)
if key & 0xFF == ord('q'):
    break

```

Sau phần xử lý phía trên, phần này ta chỉ đóng khung các biển báo đã được nhận dạng dùng hàm imshow() để hiển thị output cho người dùng.

2.7 Kết quả thực nghiệm

Đường link video:

+ Video_output1:

<https://drive.google.com/file/d/1UV7E528-a-xWvLXZfBa7wo70BzeiVbtk/view?usp=sharing>

+ Video_output2:

https://drive.google.com/file/d/1AhPWUdrT_nqvWa56NgC81qiWfu_S_lvN/view?usp=sharing

Video 1:



Figure 10. Video 1 frame 1



Figure 11. Video 1 frame 2



Figure 12. Video 1 frame 3



Figure 13. Video 1 frame 4



Figure 14. Video 1 frame 5



Figure 15. Video 1 frame 6



Figure 16. Video 1 frame 7



Figure 17. Video 1 frame 8

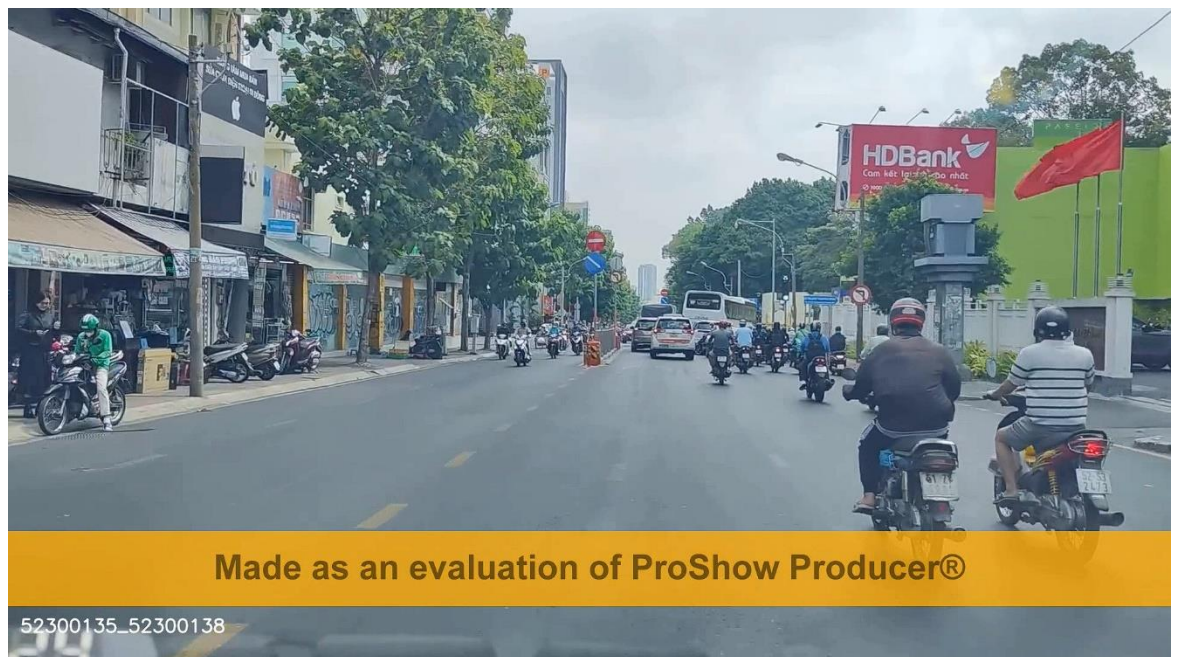


Figure 18. Video 1 frame 9



Figure 19. Video 1 frame 10

Video 2:



Figure 20. Video 2 frame 1

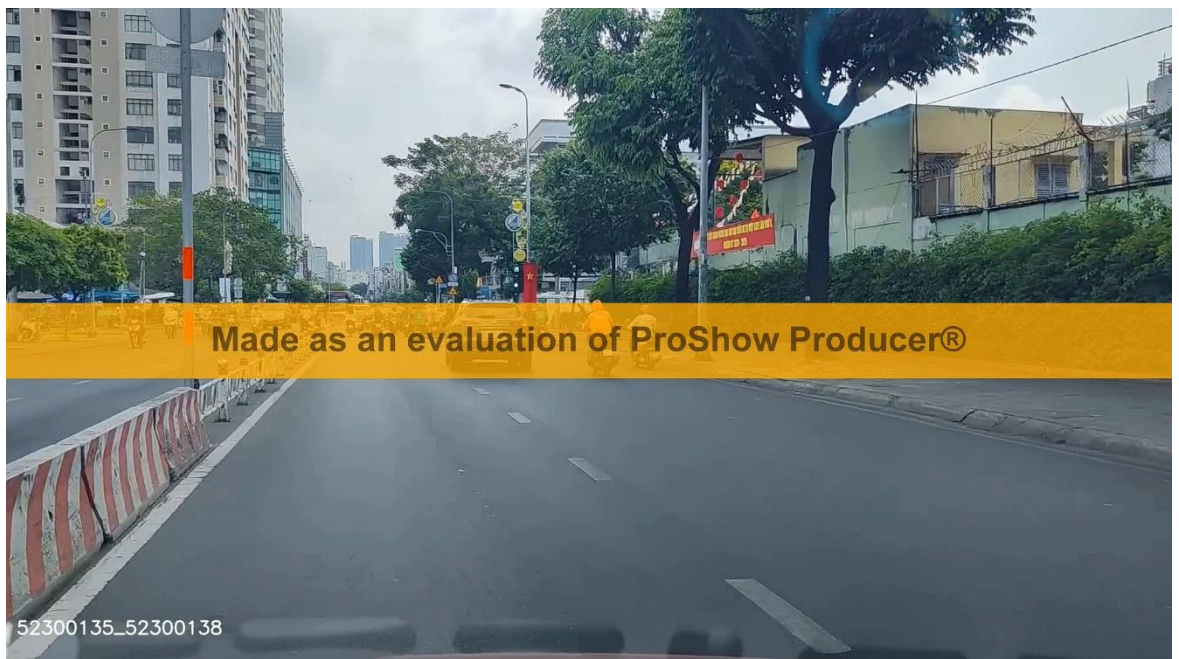


Figure 21. Video 2 frame 2

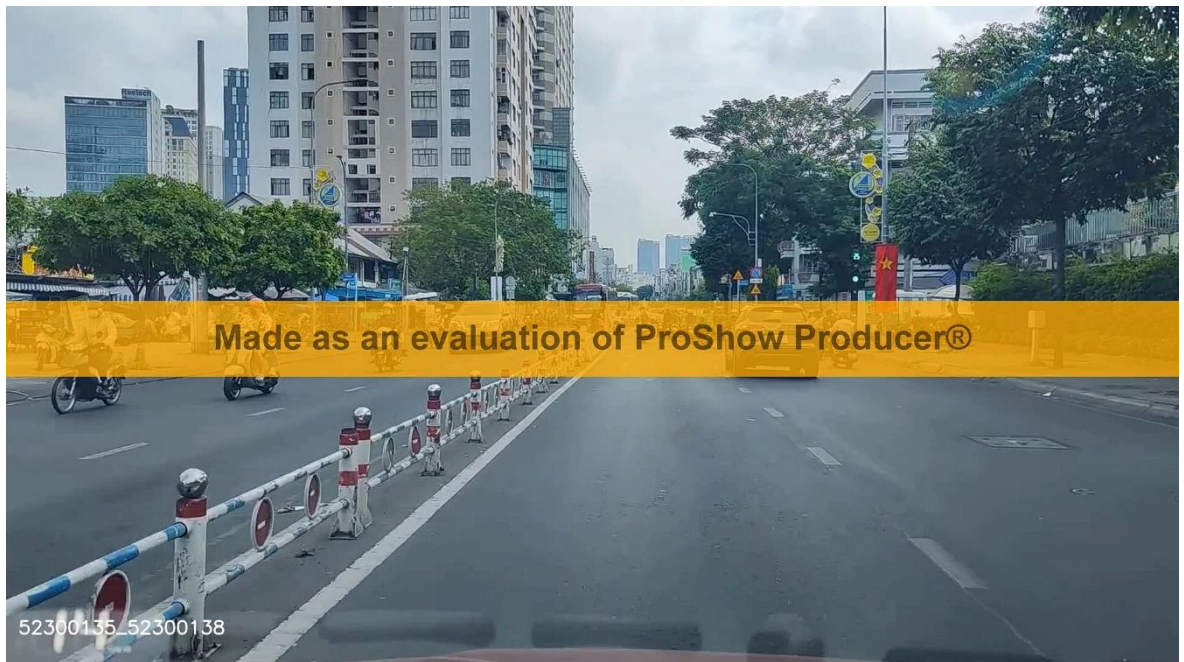


Figure 22. Video 2 frame 3



Figure 23. Video 2 frame 4



Figure 24. Video 2 frame 5



Figure 25. Video 2 frame 6

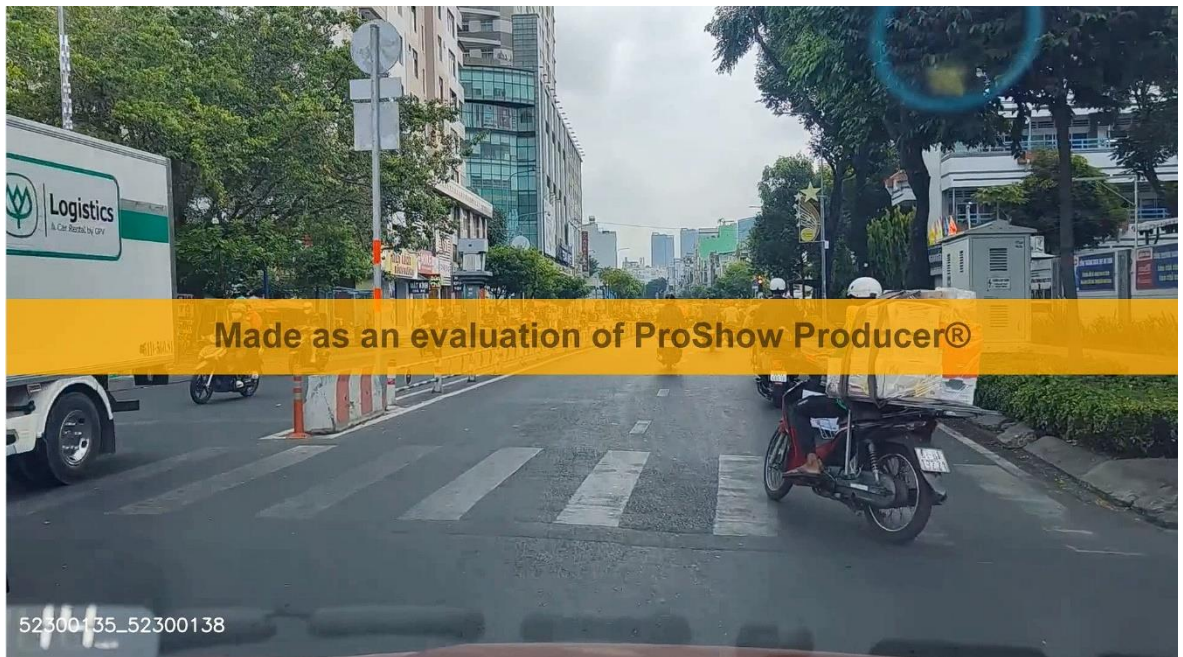


Figure 26. Video 2 frame 7



Figure 27. Video 2 frame 8

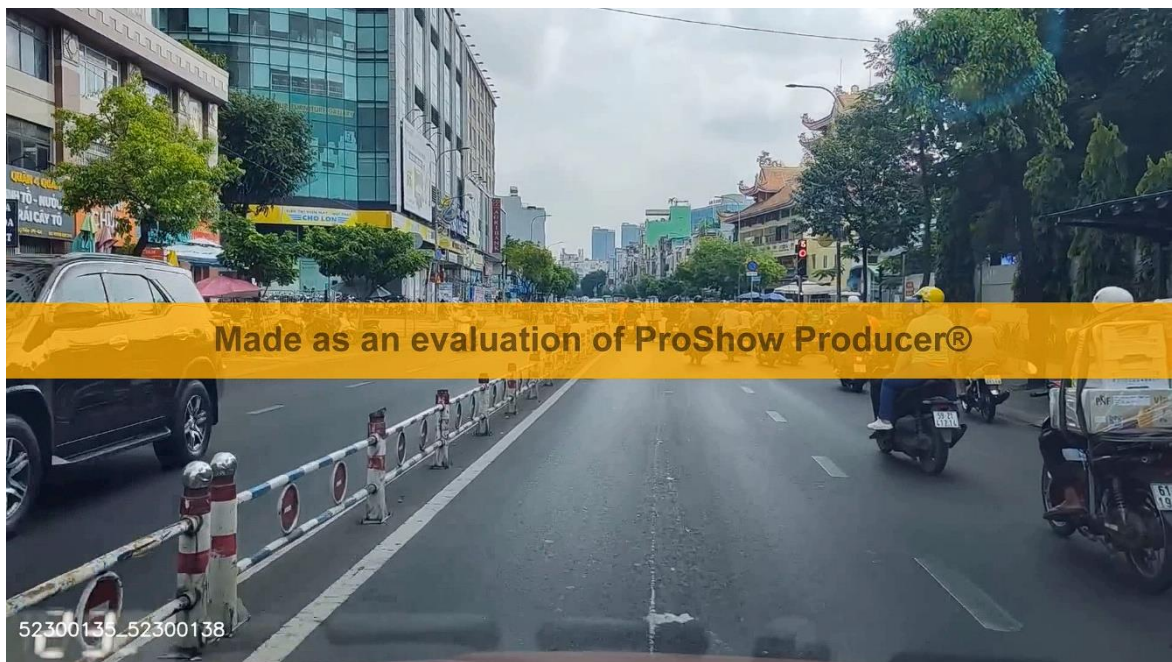


Figure 28. Video 2 frame 9



Figure 29. Video 2 frame 10

CHƯƠNG 3. Kết luận và hướng phát triển

3.1 Những điều chưa đạt được trong đề tài

Qua thời gian nghiên cứu và phát triển, vẫn có một vài điều trong đề tài lần này mà chúng em chưa đạt được, có thể kể ra như sau:

- Một số biển báo vẫn bị nhận dạng sai hoặc không nhận dạng được.
- Chưa theo dõi ổn định theo thời gian.
- Dùng hàm circularity để check không đảm bảo được việc chỉ lấy các vật thể tròn và tam giác.

3.2 Những điều đã đạt được trong đề tài

Mặc dù còn tồn tại nhiều mặt hạn chế, nhưng đề tài có đạt được một vài điều sau:

- Xây dựng thành công hệ thống phát hiện và nhận dạng biển báo giao thông theo video với luồng hoạt động rõ ràng.
- Ứng dụng đa kỹ thuật xử lý ảnh trong OpenCV.
- Khả năng mở rộng

3.3 Kết luận

Đề tài đã được thực hiện theo các nguyên tắc được đặt ra, tuy còn nhiều thiếu sót, nhưng chúng em đã có những nghiên cứu và những hướng phát triển cho đề tài của mình, áp dụng những gì được học trên lớp và tự học vào đề tài theo cách hiểu của cá nhân.

TÀI LIỆU THAM KHẢO

Timothy M. “What is Feature Matching?,” Roboflow Blog, Oct 30, 2024.
[Online]. Available: <https://blog.roboflow.com/what-is-feature-matching/>

Brown University, “CS143 Project 2: Local Feature Matching”, 2013.
Available: <https://cs.brown.edu/courses/cs143/2013/results/proj2/rroelke/>